

# PULSE Synthetic Dataset

## File guide: what each file contains and what to do with it next

Team Fluxora, Smart India Hackathon 2026 (SIH26120) | Data version 1.4, case baghewala, seed 42, noise level 0.5 | 4 October 2026

### Read this first

- **The data is synthetic.** A physics-based generator, calibrated to published Baghewala figures, produced it. No Oil India well data was used, so models learn the generator's assumptions, not real well behaviour.
- **Size:** 300 wells, 2,739 CSS cycles, 599,522 daily rows, 613 failures. All 51 automated checks pass, none fail.
- **Three rules for everyone:** (1) split train and test by **well**, never by row; (2) never use `condition_label` or the failure log as input features; (3) never use `reservoir_temp_c` or `oil_viscosity_cp` to predict production rate.
- **No dynamometer cards exist in this dataset**, so a card-based CNN cannot be trained from these files.

## 1. The files at a glance

Eight files are being shared. The first four are the data; the rest describe, check or benchmark it. The generator code that produced them is a separate zip and is only needed to regenerate or change the data.

| File                                   | Size                   | What it is                                                             | Who needs it                   | First thing to do                                  |
|----------------------------------------|------------------------|------------------------------------------------------------------------|--------------------------------|----------------------------------------------------|
| <code>css_cycle_log.csv</code>         | 41,057 rows<br>4 MB    | Steaming settings for every cycle (injection days only)                | CSS optimizer; thermal model   | Collapse to one row per well-cycle                 |
| <code>production_history.csv</code>    | 599,522 rows<br>115 MB | Daily oil, water, temperature, viscosity, plus the clean physics prior | Thermal model; CSS optimizer   | Keep production days; compute observed minus prior |
| <code>srp_vfd_data.csv</code>          | 599,522 rows<br>104 MB | Daily pump speed, load and condition signals                           | SPM recommender; early warning | Keep days with <code>pump_running = 1</code>       |
| <code>rod_failure_log.csv</code>       | 613 rows<br>18 KB      | One row per rod or pump failure                                        | Early warning                  | Turn into a 'failure within 14 days' label         |
| <code>generation_config.json</code>    | 3 KB                   | Every setting used to generate the data                                | Everyone (reference)           | Store it next to the data and cite it              |
| <code>validation_report.csv</code>     | 62 rows                | Results of the automated checks                                        | Reviewers, project lead        | Read the INFO lines                                |
| <code>validation_plots.png</code>      | image                  | Example cycles of one well                                             | Everyone                       | Use it as a plausibility reference                 |
| <code>early_warning_report.json</code> | 2 KB                   | Baseline early-warning scores on held-out wells                        | Early-warning owner            | Treat as the benchmark to beat                     |

## 2. File by file

### 2.1 css\_cycle\_log.csv: the CSS controls

One row per injection day. Settings are constant within a cycle, so a cycle with 14 injection days has 14 identical rows apart from day.

| Column                 | Meaning                                                                            | Range or unit                |
|------------------------|------------------------------------------------------------------------------------|------------------------------|
| well_id, cycle         | Well (WELL-001 to WELL-300) and its steaming cycle                                 | cycle 1 to 10                |
| day                    | Days since the well's first steaming; continuous across cycles                     | integer                      |
| phase                  | Always injection in this file                                                      |                              |
| steam_volume_m3        | Steam injected per day, cold-water equivalent                                      | 150 to 250 m <sup>3</sup> /d |
| steam_quality          | Fraction of the steam that is vapour                                               | 0.45 to 0.70                 |
| injection_temp_c       | Steam temperature                                                                  | 302 to 330 C                 |
| injection_pressure_kpa | Injection pressure; equals the saturation pressure of the temperature (within 4 %) | 8,800 to 13,300 kPa          |
| soak_time_days         | Shut-in days planned after injection                                               | 2 to 14 d                    |

#### Do next

1. Group by well\_id, cycle to get one row per cycle: steam rate, quality, temperature, soak days, injection days (the row count) and total steam (rate x days).
2. Join the cycle totals from production\_history.csv to get oil per cycle-day and steam-oil ratio (code in section 3).
3. Use these cycle rows as the inputs of the CSS optimizer, and as context features for the thermal-viscosity residual model.

Watch out: pressure and temperature move together, so treat them as one variable. Pick one for the optimizer.

### 2.2 production\_history.csv: daily results and the physics prior

One row per day from the start of soak to the end of the cycle, for each well-cycle. Phases: 572,830 production days, 22,075 soak days, 4,617 workover days.

| Column                                                                                             | Meaning                                                       | Notes                                              |
|----------------------------------------------------------------------------------------------------|---------------------------------------------------------------|----------------------------------------------------|
| well_id, cycle, day, phase                                                                         | Keys and phase: soak, production or workover                  | Same keys as the pump file                         |
| days_since_soak_start                                                                              | Days since the cycle's soak began                             | 0 on the first soak day                            |
| oil_rate_bbl_day, gross_liquid_bbl_day                                                             | Observed oil and total liquid lifted                          | bbl/d. Exactly 0 during soak and workover          |
| water_cut_pct                                                                                      | Observed water cut                                            | Empty (not 0) during soak and workover             |
| reservoir_temp_c, oil_viscosity_cp                                                                 | Heated-zone average temperature and oil viscosity, with noise | <b>Latent model states, not field measurements</b> |
| prior_oil_rate_bbl_day, prior_gross_liquid_bbl_day, prior_reservoir_temp_c, prior_oil_viscosity_cp | The clean physics-model prediction for the same day           | Observed minus prior is the residual to learn      |

#### Do next

1. Filter phase == 'production' for rate models. Soak and workover rows are zeros by design.
2. **Thermal-viscosity residual model:** target = oil\_rate\_bbl\_day - prior\_oil\_rate\_bbl\_day (for viscosity, observed / prior - 1). Features: the prior\_\* columns, days\_since\_soak\_start, cycle number and that cycle's controls.
3. **CSS optimizer targets:** sum oil per cycle (x 0.158987 to convert bbl to m<sup>3</sup>), divide by cycle days, and compute SOR = total steam / oil. About 3.2 % of cycles have SOR above 10 (wells near their economic limit); flag or drop them.
4. Score every model on **held-out wells**.

Watch out: do not feed `reservoir_temp_c` or `oil_viscosity_cp` into a model that predicts rate. They directly set the rate. A real deployment would estimate them with the thermal model.

## 2.3 `srp_vfd_data.csv`: pump operation and load signals

Same 599,522 rows and keys as the production file (`well_id`, `cycle`, `day`), so the two join one-to-one. All pump columns are 0 while the pump is off (soak and workover).

| Column                                                                            | Meaning                                                                            | Range or unit                         |
|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------|---------------------------------------|
| <code>pump_running</code>                                                         | 1 when the pump is operating                                                       | 0 or 1                                |
| <code>spm</code> , <code>stroke_length_in</code> , <code>vfd_frequency_hz</code>  | Strokes per minute, stroke length, drive frequency                                 | 2.0 to 9.6 SPM; 96 in; 12.5 to 60 Hz  |
| <code>motor_load_pct</code>                                                       | Motor load                                                                         | 0 to 100 % (clipped at 100)           |
| <code>pump_fillage_pct</code>                                                     | How full the pump barrel is                                                        | 0 to 100 %                            |
| <code>rod_floating_index</code>                                                   | Pump speed over the viscosity-limited rod-fall speed; above 1 means the rods float | 0 to about 3.7                        |
| <code>polished_rod_peak_load_kn</code> ,<br><code>polished_rod_min_load_kn</code> | Daily peak and minimum rod load (approximate, formula-based)                       | kN; the minimum drops when rods float |
| <code>load_variability_pct</code>                                                 | Daily variability of the rod load; rises before failures                           | percent                               |
| <code>condition_label</code>                                                      | normal, shut_in, workover, or the failure mode on the failure day                  | <b>Target. Never a feature</b>        |

### Do next

1. Keep rows with `pump_running == 1` (572,830 days).
  2. **Early warning:** build rolling means over 3, 7, 14 and 30 days, a 7-day trend, and exposure since the last workover (cumulative floating exposure, days with floating, days since restart); label from the failure log. `train_early_warning.py` in the generator zip does exactly this as a baseline.
  3. **SPM recommender:** this file records what simulated operators chose, not what would have happened at other speeds. For reinforcement learning, import `cycle_inflow` and `pump_response` from `generate_dataset.py` as the simulator and write the reward yourselves (oil lifted, energy, rod-floating margin).
  4. Join to `production_history.csv` on `well_id`, `cycle`, `day` when you need lifted oil next to pump settings.
- Watch out: `rod_floating_index` is computed from SPM and viscosity. A model that 'predicts' it from those inputs just relearns the formula.

## 2.4 `rod_failure_log.csv`: the failure events

One row per failure: 613 in total (374 `rod_part`, 173 `pump_unset`, 66 `worn_barrel`), about 0.39 per running well-year. After each failure the well is in workover for `downtime_days` (4 to 12), then its damage resets.

| Column                                    | Meaning                                                                                                                                         |
|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>well_id</code> , <code>cycle</code> | Where the failure happened                                                                                                                      |
| <code>failure_date_day</code>             | The day of the failure; matches day in the pump file                                                                                            |
| <code>failure_mode</code>                 | <code>rod_part</code> (impact loading from rod floating), <code>pump_unset</code> (fluid pound at low fillage), <code>worn_barrel</code> (wear) |
| <code>downtime_days</code>                | Days in workover after the failure                                                                                                              |

### Do next

1. For every running day in the pump file, set the label to 1 if a failure occurs on the same well within the next 14 days (the failure day included), else 0.
2. Drop workover days, then split train and test by well.
3. Report PR-AUC and the share of failures caught at least 3 days ahead at a fixed false-flag rate (for example 2 % of healthy days). Do not report accuracy: only 1.6 % of days are positive.

Watch out: failures are simulated. They are caused by floating, pounding and wear exposure that is visible in the pump file, and the failure rate (about 0.4 per well-year) is an assumption.

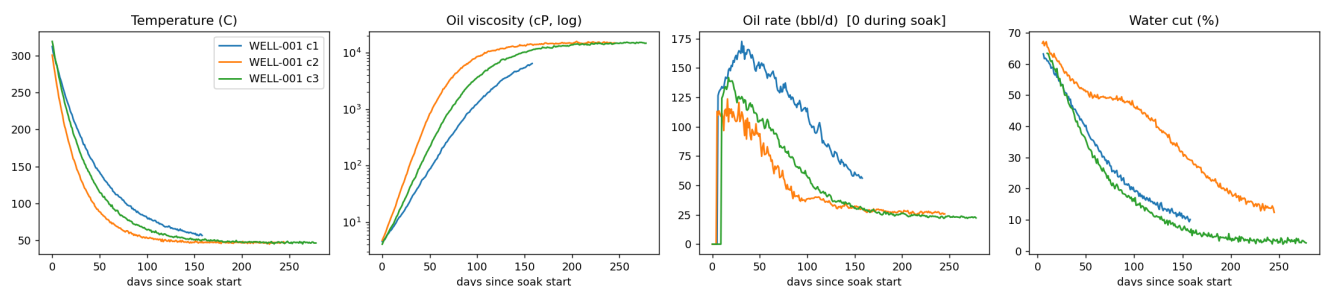
## 2.5 generation\_config.json: how the data was made

Holds the version (1.4), case (baghewala), seed (42), noise level (0.5), economic limit (stop steaming below an oil-steam ratio of 0.12), number of wells and cycles, and every model parameter (reservoir, steam ranges, pump, failure model, viscosity table).

1. Keep it in the same folder as the data and cite it in the report.
2. To rebuild identical data, run the generator with the same seed and flags (section 5).
3. To change an assumption, change the generator's parameters and regenerate. Editing this file by hand does not change the data.

## 2.6 validation\_report.csv and validation\_plots.png: the checks already run

The report has one row per check with file, check, status, detail. Result: 51 PASS, 11 INFO, 0 WARN, 0 FAIL. The checks cover structure and units, steam-table consistency, pump limits, water balance, whether the CSS settings influence oil, failure behaviour, and agreement with published figures. The plot shows temperature, viscosity, oil rate and water cut for three cycles of WELL-001.



1. Read the INFO lines: cycle SOR (median 2.56), the soak optimum, peak oil rate, failure counts, and how many wells hit the economic limit.
2. Compare your model's output curves with the plot. After a heat pulse the temperature should decay, viscosity should climb back toward about 15,000 cP, and oil rate should peak shortly after soak and then fall.
3. Re-run `validate_dataset.py` after any change to the data files.

## 2.7 early\_warning\_report.json: the baseline to beat

Baseline early-warning results on held-out wells: 14-day horizon, 199 test failures, threshold set to flag 2 % of healthy days (about 7 flagged days per running well-year).

| Model                              | ROC-AUC | PR-AUC | Failures caught 3+ days ahead |
|------------------------------------|---------|--------|-------------------------------|
| Gradient boosting, all features    | 0.98    | 0.51   | 88 %                          |
| Rule: load variability only        | 0.89    | 0.18   | 70 %                          |
| Rule: rod-floating index only      | 0.85    | 0.15   | 36 %                          |
| Rule: cumulative floating exposure | 0.81    | 0.07   | 11 %                          |

1. Use this as the benchmark: any new model should beat PR-AUC 0.51 on the same held-out wells, and should be reported next to the simple rules.
2. A random model scores a PR-AUC of about 0.017, so 0.51 is well above chance.
3. These scores are an **optimistic ceiling**: failures are caused by the same exposures used as features. To see the warning degrade, regenerate with `--precursor_scale 0.25`.

## 3. How the files connect

| Join                                  | Keys                                    | Result                                  |
|---------------------------------------|-----------------------------------------|-----------------------------------------|
| css_cycle_log with production_history | well_id + cycle                         | Controls next to results for each cycle |
| production_history with srp_vfd_data  | well_id + cycle + day                   | One-to-one: same 599,522 rows           |
| rod_failure_log with srp_vfd_data     | well_id + cycle, failure_date_day = day | Marks the failure day for labelling     |

Starter code that builds the per-cycle table and the daily pump table (tested on these files):

```
import pandas as pd
K = ["well_id", "cycle"]
css = pd.read_csv("css_cycle_log.csv")
prod = pd.read_csv("production_history.csv")
srp = pd.read_csv("srp_vfd_data.csv")
fail = pd.read_csv("rod_failure_log.csv")

# one row per well-cycle: controls and results
ctrl = css.groupby(K).agg(steam_rate=("steam_volume_m3", "first"),
                        inj_days=("day", "size"),
                        quality=("steam_quality", "first"),
                        temp_c=("injection_temp_c", "first"),
                        soak_days=("soak_time_days", "first"))
oil = (prod[prod.phase == "production"].groupby(K)
      .oil_rate_bbl_day.sum() * 0.158987).rename("oil_m3")
cycles = ctrl.join(oil)
cycles["sor"] = cycles.steam_rate * cycles.inj_days / cycles.oil_m3

# daily table for the pump: same rows as production, more columns
daily = srp.merge(prod.drop(columns="phase"), on=K + ["day"])
daily = daily[daily.pump_running == 1]
```

Tip: the two big files are about 100 MB each. Read only the columns you need with `usecols=[...]`, or work on a subset of wells.

## 4. Which model uses which file

| Model                                              | Files                                                     | Target                                 | Split and caveats                                                                                                                         |
|----------------------------------------------------|-----------------------------------------------------------|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Thermal-viscosity residual (GBM)                   | production_history + css_cycle_log                        | Observed minus prior (rate, viscosity) | By well. The prior is the generator's own physics, so the GBM learns the injected deviations                                              |
| CSS parameter optimizer                            | css_cycle_log + production_history                        | Oil per cycle-day, SOR                 | By well. Control for cycle number; flag SOR above 10. Steam volume and days show clearly; temperature, quality and soak are small effects |
| SPM / stroke recommender                           | srp_vfd_data + production_history (+ generator functions) | A reward you define                    | By well. Do not imitate the logged SPM: simulated operators chose it                                                                      |
| Early warning (no cards)                           | srp_vfd_data + rod_failure_log                            | Failure within 14 days                 | By well. Features from running days only; compare against the rule baselines                                                              |
| Dynamometer CNN, card-based rod-floating detection | none                                                      | n/a                                    | <b>Not possible:</b> there are no dynamometer cards in this dataset                                                                       |

## 5. Regenerating the data

The same data can be rebuilt from the generator zip in about a minute. Do not mix files from different seeds or versions.

```
pip install -r requirements.txt
python generate_dataset.py --n_wells 300 --n_cycles 10 --noise_level 0.5 \
    --skip_cards --seed 42 --out_dir synthetic_data
python validate_dataset.py --data_dir synthetic_data
python train_early_warning.py --data_dir synthetic_data --horizon 14
```

- **New random data:** change `--seed`. **Harder early warning:** add `--precursor_scale 0.25`. **No economic limit:** `--min_osr 0`.
- **The warning script stops with 'too few positive rows':** use at least 150 wells and 10 cycles.
- **The validator reports a FAIL:** the files were edited or come from another version; regenerate them.

## 6. What to say about this data

Suggested wording: "Calibrated to published Baghewala facts and analog CSS papers; the pipeline is built to retrain on real Oil India data." The data matches published field-level figures on average, but every well-level number is a modelling assumption.

| Calibration check                         | Published figure                | This data |
|-------------------------------------------|---------------------------------|-----------|
| Field-average oil rate per producing well | about 36 bbl/d (range 25 to 50) | 34 bbl/d  |
| Stimulated vs cold rate                   | about 3x (range 2 to 8)         | 2.8x      |
| Cycle oil-steam ratio                     | 0.19 to 0.64                    | 0.39      |
| Cycle-average water cut                   | 46 to 74 %                      | 61 %      |

- **Assumed, not sourced:** reservoir pressure (8 MPa), net pay (20 m), effective permeability (6,500 mD, tuned to the field-average rate), the hot-end viscosity, and the whole failure model (about 0.4 failures per well-year, precursor shape and strength).
- **Weak sources:** the 36 bbl/d figure comes from a press report of an Oil India number; the 5 to 6 times pilot response comes from an unverified document.
- **Early-warning scores are an optimistic ceiling**, because failures are caused by the exposures used as features.
- **Do not claim** card-based detection or validation on real Baghewala data.
- Every source and its reliability grade is listed in `calibration.py` and the README inside the generator zip.